Exp: 6 Error Detection and correction using Hamming Code

Aim:

To write a program to implement error detection and correction using Hamming code.

Algorithm

1. Start the program.
2. Read the number of 4-bit data blocks (n) forming the input data stream
3. Simulate transmission by introducing a single-bit error at a chosen bit position in the code word.
4. At the receiver, recompute the check bits from the received code word:

$C1 = P1 \text{ XOR } d1 \text{ XOR } d2 \text{ XOR } d4$

$C2 = P2 \text{ XOR } d1 \text{ XOR } d3 \text{ XOR } d4$

$C4 = P4 \text{ XOR } d2 \text{ XOR } d3 \text{ XOR } d4$

5. Compute Error position = C4C2C1
6. Extract the original 4 data bits from the corrected 7-bit code word.
8. Repeat steps 3 to 8 for all n blocks in the data stream.
9. Stop the program

Program:

```
def encode(d):
    """d : list of 4 bits [d1,d2,d3,d4] → returns 7 bit hamming code list
    positions (1-indexed : 1=p1
    2=p2  3=d1  4=p4  5=d2  6=d3  7=d4"""
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    if error_pos != 0:
        print(f"error detected at bit position: {error_pos}
        -> correcting ....")
    else:
        print("no error detected.")
    return error_pos

def main():
    print("=== HAMMING CODE - ERROR DETECTION
    AND CORRECTION ===")
    n = int(input("Enter number of 4-bit data
        blocks in the stream : "))
```

output :

=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 11-bit data blocks in the stream : 2

--- Block 1 ---

Enter 4 data bits (space separated, eg 1011) : 1011

Original data Bits : 1011

Encoded Hamming code (7 bits) : 0110011

Enter bit position to introduce error (1-7, 0 = no error) : 3

Transmitted code (with error induced at pos 3) : 0100011

Received code : 0100011

Checking for errors...

Error detected at bits position : 3 → correcting...

Corrected code : 0110011

Recovered Data Bits : 1011

--- Block 2 ---

Enter 4 data bits (space separated, eg : 1011) : 0110

Original Data Bits : 0110

Encoded Hamming code (7 bits) : 1100110

Enter bit position to introduce error (1-7, 0 = no error) : 6

Received code : 1100100

Checking for errors...

Error detected at bit position : 6 → correcting...

Corrected code : 1100110

Recovered Data Bits : 0110

```
for b in range(1, n+1):
    print(f"\n--- Block {b} ---")
    d = list(map(int, input("Enter 4 data bits (space separated, e.g., 1 0 1 1): ").split()))
    h = encode(d)
    print(f"Original Data Bits : {to_str(d)}")
    print(f"Encoded Hamming code (7 bits): {to_str(h)}")
    if 1 <= pos <= 7:
        h[pos - 1] ^= 1
        print(f"Transmitted code (with error induced at pos {pos}): {to_str(h)}")
    else:
        print(f"Transmitted code (no error induced): {to_str(h)}")
    print(f"Received code : {to_str(h)}")
    print("Checking for errors...")
    detect_and_correct(h)
    print(f"Corrected code : {to_str(h)}")
    rd = [h[2], h[4], h[5], h[6]]
    print(f"Recovered Data Bits: {to_str(rd)}")
if __name__ == "__main__":
    main()
```

Result:

The program to implement error detection and correction using Hamming code concept was successfully written in python, executed and verified using a sample input data stream.